

Practical Experiment 4

Question 1:

Code:

```
#include <stdio.h>

#include <unistd.h>

#include <sys/wait.h>

int main()
{
    char command[50];

    printf("Enter command: ");
    scanf("%s", command);

    int pid = fork();

    if (pid == 0)
    {
        printf("Child PID: %d\n", getpid());
        execlp(command, command, NULL);
    }
```

```

else

{

    printf("Parent PID: %d\n", getpid());

    printf("Child PID: %d\n", pid);

    wait(NULL);

    printf("Child completed\n");

}

return 0;

}

```

Output :

```

teja27@TEJA:~$ nano exp4-1.c
teja27@TEJA:~$ gcc exp4-1.c -o exp4-1
teja27@TEJA:~$ ./exp4-1
Enter command: ls
Parent PID: 563
Child PID: 566
Child PID: 566
Experiment1_old_backup      command_executor  exp4-1.c      main      parentprocess  prog1.c  prog4.c  prog8.c      systemcalls
OSSP                        command_executor.c  fork          main.c    parentprocess.c  prog2.c  prog5.c  prog9.c      systemcalls.c
OSSP_All_4_Experiments.pdf  exp1.c.save       fork.c        myshell   pc.c.save       prog2.c  prog5.c  prog9.c      week2.c
OSSP_All_4_Experiments.pdf:Zone.Identifier  exp3              fork_exec     notes.txt  prog-6          prog3    prog7    projects    world.c
Team-19_OSSP               exp3.c            fork_exec.c   ossp      prog-6.c        prog3.c  prog7.c  readsystemcalls
a.out                      exp4-1            input.txt     osspp     prog1            prog4    prog8    readsystemcalls.c
Child completed
teja27@TEJA:~$ |

```

Question 2:

Code:

```

#include <stdio.h>

#include <unistd.h>

#include <sys/wait.h>

```

```
int main()

{

    char command[50];


    printf("Enter command: ");
    scanf("%s", command);


    int pid = fork();


    if (pid == 0)
    {
        printf("Child PID: %d\n", getpid());
        execlp(command, command, NULL);
    }
    else
    {
        printf("Parent PID: %d\n", getpid());
        printf("Child PID: %d\n", pid);


        wait(NULL);


        printf("Child completed\n");
    }


    return 0;
```

```
}
```

Output:

```
teja27@TEJA:~$ gcc exp4-2.c -o exp4-2
teja27@TEJA:~$ ./exp4-2
Enter command: pwd
Parent PID: 691
Child PID: 696
Child PID: 696
/home/teja27
Child completed
teja27@TEJA:~$ |
```

Question 3:

Code:

```
#include <stdio.h>
```

```
#include <fcntl.h>
```

```
#include <unistd.h>
```

```
int main()
```

```
{
```

```
    int source, destination;
```

```
    char buffer[100];
```

```
    int n;
```

```
    source = open("source.txt", O_RDONLY);
```

```
    destination = open("copy.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

```
    n = read(source, buffer, sizeof(buffer));
```

```
write(destination, buffer, n);

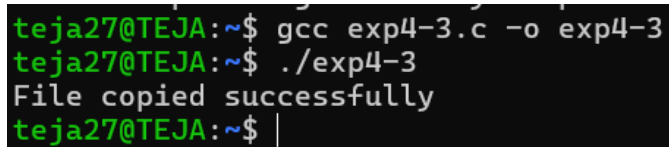
close(source);

close(destination);

printf("File copied successfully\n");

return 0;
}
```

Output:

A terminal window with a black background and green text. The prompt is 'teja27@TEJA:~\$'. The first command is 'gcc exp4-3.c -o exp4-3'. The second command is './exp4-3'. The output is 'File copied successfully'. The prompt is now 'teja27@TEJA:~\$ |' with a cursor.

```
teja27@TEJA:~$ gcc exp4-3.c -o exp4-3
teja27@TEJA:~$ ./exp4-3
File copied successfully
teja27@TEJA:~$ |
```

Question 4:

Code:

```
#include <stdio.h>

#include <unistd.h>

#include <sys/wait.h>

int main()
{
    int pid = fork();
```

```
if (pid == 0)
{
    printf("Child Process\n");
    printf("PID: %d\n", getpid());
    printf("PPID: %d\n", getppid());

    sleep(5);

    printf("Child completed\n");
}
else
{
    printf("Parent Process\n");
    printf("PID: %d\n", getpid());
    printf("Child PID: %d\n", pid);

    wait(NULL);

    printf("Parent completed\n");
}

return 0;
}
```

Output:

```
teja27@TEJA:~$ gcc exp4-4.c -o exp4-4
teja27@TEJA:~$ ./exp4-4
Parent Process
PID: 813
Child PID: 814
Child Process
PID: 814
PPID: 813
Child completed
Parent completed
teja27@TEJA:~$ |
```